

DevOps Weather App Deployment Project

End-to-End CI/CD | Infrastructure as Code | Cloud Deployment | Kubernetes | Monitoring



Submitted By:
Kusukuntla Vyshnavi
12303433
Roll no. 02

1. Project Overview

This project demonstrates a complete end-to-end DevOps implementation using industry-standard cloud technologies and automation tools. The application is a React-based Weather App that is fully automated from source code to cloud deployment and real-time monitoring.

The primary goal is to simulate a real-world DevOps workflow where every step from a developer pushing code to the application being live on the cloud is handled automatically without manual intervention.

Key Capabilities

- Automated Infrastructure Provisioning using Terraform
- CI/CD Pipeline using Jenkins with GitHub Webhook integration
- Containerization using Docker with multi-stage builds
- Container Orchestration using K3s (Lightweight Kubernetes)
- Configuration Management using Ansible
- Cloud deployment on AWS EC2
- Real-Time Monitoring with Prometheus, Grafana, and Node Exporter

2. Technologies Used

The following tools and technologies form the core of this project. Each tool plays a specific role in the automation pipeline.

Tool / Technology	Purpose
Git	Distributed version control for tracking source code changes
GitHub	Cloud-based repository hosting and webhook trigger source
Docker	Containerization platform for packaging the React app
Jenkins	Open-source CI/CD server for pipeline automation
Terraform	Infrastructure as Code tool for provisioning AWS resources
Ansible	Agentless configuration management and automation tool
AWS EC2	Cloud virtual machine to host the application and monitoring stack
Prometheus	Time-series metrics collection and alerting engine
Grafana	Interactive visualization dashboard for monitoring data
Node Exporter	Exposes Linux system metrics for Prometheus to scrape
K3s	Lightweight Kubernetes distribution for container orchestration
ngrok	Tunneling tool to expose local Jenkins to the internet for webhooks

3. Project Architecture

The architecture follows a linear CI/CD pipeline where every stage is triggered automatically by the previous one. The flow begins with a developer's Git push and ends with a fully monitored, Kubernetes-managed application running on AWS.

Pipeline Flow

Step 1 — Developer

Developer writes code locally and pushes changes to the GitHub repository.

Step 2 — GitHub Webhook

GitHub detects the push event and fires a webhook to the Jenkins server (exposed via ngrok).

Step 3 — Jenkins CI/CD

Jenkins receives the webhook and automatically triggers the pipeline defined in the Jenkinsfile.

Step 4 — Terraform Provisioning

Jenkins runs Terraform to provision a new AWS EC2 instance along with security groups and key pairs.

Step 5 — Ansible Configuration

Once EC2 is ready, Ansible connects via SSH and installs Docker, K3s, Prometheus, Grafana, and Node Exporter.

Step 6 — Docker + K3s Deployment

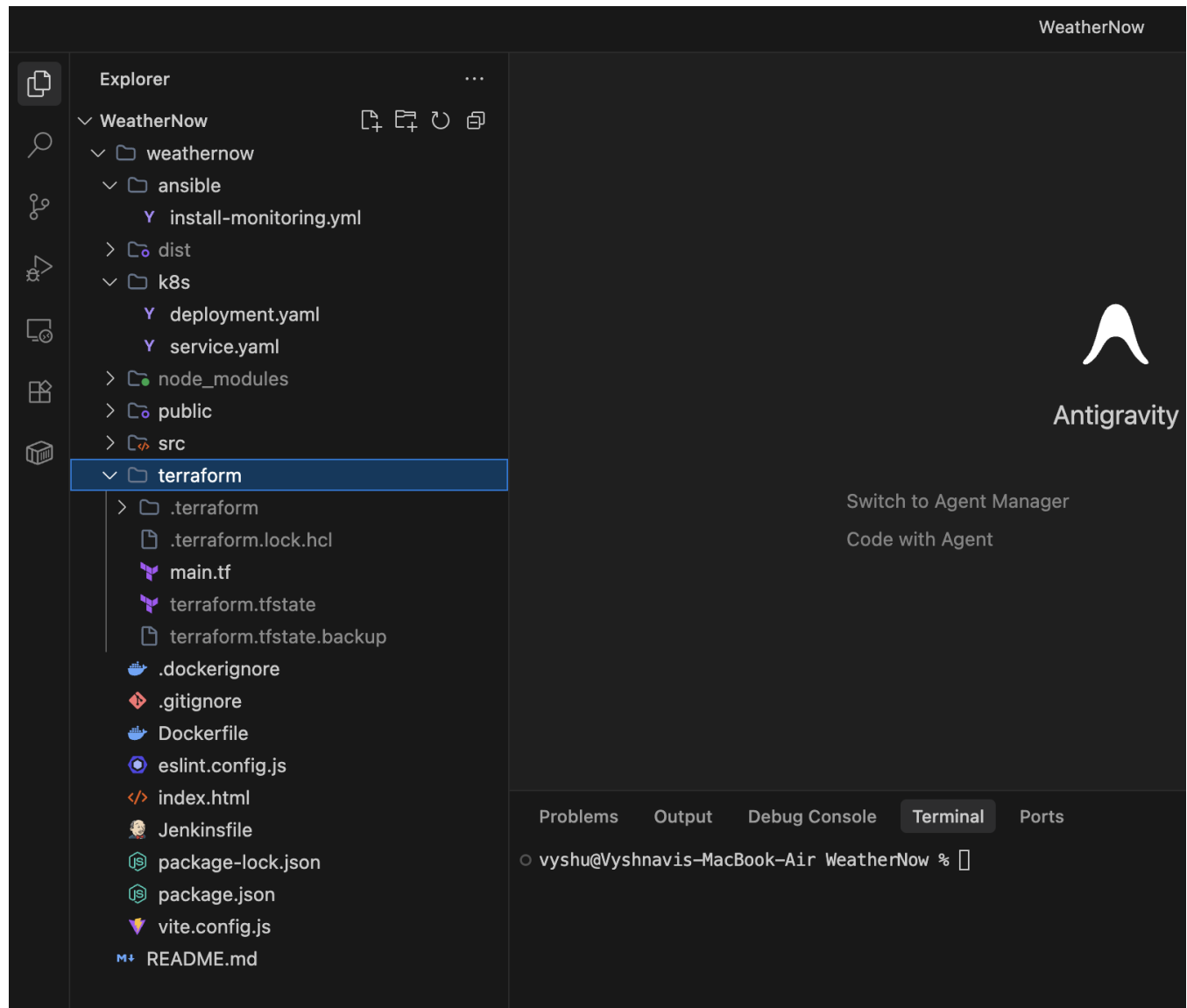
The Weather App Docker image is pulled from Docker Hub and deployed as Kubernetes pods on K3s.

Step 7 — Monitoring Live

Prometheus begins scraping metrics from Node Exporter. Grafana dashboards become available automatically.

4. Project Structure

Below is the complete folder structure of the project repository:



5. Prerequisites & Local Setup

Before starting, the following tools must be installed on the local development machine (macOS assumed, using Homebrew).

5.1 Install Git

```
brew install git
git --version
```

5.2 Install Docker

Download and install Docker Desktop from the official site:

```
https://www.docker.com/products/docker-desktop/
docker --version
```

5.3 Install Terraform

```
brew install terraform
terraform -version
```

5.4 Install Jenkins

```
brew install jenkins-lts
brew services start jenkins-lts
```

Access Jenkins at: <http://localhost:8080>

5.5 Install Ansible

```
brew install ansible
ansible --version
```

5.6 Install ngrok

```
brew install ngrok/ngrok/ngrok
ngrok config add-authtoken YOUR_TOKEN
ngrok http 8080
```

5.7 Configure AWS CLI

```
brew install awscli
aws configure
```

Enter: AWS Access Key, AWS Secret Key, Region (us-east-1), Output format (json)

6. Docker — Containerization

Docker is used to package the React Weather App into a portable container image. A multi-stage build is used to keep the final image small and production ready.

Dockerfile

```
# Multi-stage build for React app
FROM node:18-alpine AS build

# Set working directory
WORKDIR /app

# Copy package files
COPY package*.json ./

# Install dependencies
RUN npm install

# Copy source code
COPY ..

# Build the app
RUN npm run build

# Production stage with nginx
FROM nginx:alpine

# Copy built files from build stage
COPY --from=build /app/dist /usr/share/nginx/html

# Expose port 80
EXPOSE 80

# Start nginx
CMD ["nginx", "-g", "daemon off;"]
```

Build and Push to Docker Hub

```
docker build -t vyshnavil411/weather-app .
docker push vyshnavil411/weather-app
```

The image is stored on Docker Hub so that Ansible and Kubernetes can pull it during deployment on the EC2 instance.

7. Terraform — Infrastructure as Code

Terraform is used to define and provision AWS cloud infrastructure in a declarative and version-controlled manner. This eliminates manual AWS Console operations.

Resources Provisioned

- AWS EC2 instance (Ubuntu, t2.micro or t2.small)
- Security Group with all required inbound/outbound rules
- SSH Key Pair for Ansible and Jenkins access
- Elastic IP (optional) for stable public access

Terraform Workflow

```
terraform init      # Download providers and modules
terraform validate  # Check configuration syntax
terraform plan      # Preview infrastructure changes
terraform apply     # Provision infrastructure on AWS
terraform destroy   # Tear down all resources (manual approval)
```

Security Group Port Configuration

Port	Service
22	SSH — Remote access for Ansible and debugging
80	HTTP — Weather App served by Nginx
3000	Grafana — Monitoring visualization dashboard
8080	Jenkins — CI/CD server (local only, via ngrok)
9090	Prometheus — Metrics collection server
9100	Node Exporter — System metrics endpoint
30080	K3s NodePort — Kubernetes app access

8. Jenkins — CI/CD Pipeline

Jenkins serves as the automation backbone of this project. The pipeline is defined in a Jenkinsfile committed to the repository, enabling pipeline-as-code.

Pipeline Stages

1. **Terraform Init** — Initializes Terraform working directory and downloads the AWS provider plugin.
2. **Terraform Validate** — Validates the Terraform configuration files for syntax and logical errors.
3. **Terraform Plan** — Shows a preview of infrastructure changes before applying them.
4. **Terraform Apply** — Provisions the EC2 instance and security group on AWS automatically.
5. **Generate Ansible Inventory** — Dynamically writes the EC2 public IP into the Ansible inventory file.
6. **Wait for EC2** — Polls the EC2 instance until SSH becomes available (typically 30–60 seconds).
7. **Install Monitoring Stack** — Runs the Ansible playbook to install Docker, K3s, app, Prometheus, Grafana, and Node Exporter.
8. **Destroy Infrastructure** — Requires manual approval in Jenkins before tearing down all AWS resources.

GitHub Webhook Configuration

The webhook connects GitHub push events directly to Jenkins, enabling fully automatic pipeline triggers.

```
GitHub Repo > Settings > Webhooks > Add Webhook
Payload URL: https://YOUR-NGROK-URL/github-webhook/
Content Type: application/json
Events: Just the push event
```

9. Ansible — Configuration Management

Ansible is used for agentless configuration of the EC2 instance. It connects via SSH and runs a playbook that fully prepares the server with all required software.

What Ansible Installs

- Docker — container runtime for building and running images
- K3s — lightweight Kubernetes for orchestrating app pods
- Weather App — deploys Kubernetes manifests (deployment + service)
- Prometheus — sets up metrics scraping with proper configuration
- Node Exporter — starts the system metrics exporter on port 9100
- Grafana — installs and auto-provisions the dashboard (ID: 1860)

Install K3s via Ansible

```
- name: Install K3s
  shell: |
    curl -sfL https://get.k3s.io | sh -
```

Deploy Kubernetes Manifests via Ansible

```
- name: Deploy Kubernetes manifests
  shell: |
    sudo kubectl apply -f /home/ubuntu/weather-app/weathernow/k8s/
```

10. Kubernetes with K3s

K3s is a lightweight, CNCF certified Kubernetes distribution designed for resource-constrained environments. It provides full Kubernetes functionality while running efficiently on a single t2.micro EC2 instance.

Why K3s?

- Significantly lower memory footprint than full Kubernetes
- Single binary installation no complex kubeadm setup
- Full Kubernetes API compatibility
- Includes built-in container runtime (containerd)
- Auto-healing: pods restart automatically if they crash

Kubernetes Deployment Manifest

```
apiVersion: apps/v1
kind: Deployment

metadata:
  name: weather-app

spec:
  replicas: 2

  selector:
    matchLabels:
      app: weather-app

  template:
    metadata:
      labels:
        app: weather-app

    spec:
      containers:
        - name: weather-app
          image: vyshnavi1411/weather-app:latest

      ports:
        - containerPort: 80
```

Kubernetes Service Manifest

```
apiVersion: v1
kind: Service

metadata:
  name: weather-service
```

DevOps Weather App Deployment Project

```
spec:
  type: NodePort

selector:
  app: weather-app

ports:
  - port: 80
  targetPort: 80
  nodePort: 30080
```

Verify the Kubernetes Cluster

```
ssh ubuntu@PUBLIC-IP
sudo kubectl get nodes
sudo kubectl get pods
sudo kubectl get svc
```

Verification of Kubernetes Cluster

```
ubuntu@ip-172-31-47-206:~$ sudo kubectl get nodes
NAME                STATUS    ROLES    AGE     VERSION
ip-172-31-47-206    Ready    control-plane  9m56s   v1.35.4+k3s1
ubuntu@ip-172-31-47-206:~$ sudo kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
weather-app-dc584544c-6m5hn        1/1     Running   0           9m9s
weather-app-dc584544c-tkcpf        1/1     Running   0           9m9s
ubuntu@ip-172-31-47-206:~$ sudo kubectl get svc
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
kubernetes          ClusterIP   10.43.0.1     <none>         443/TCP          10m
weather-service     NodePort    10.43.129.250 <none>         80:30080/TCP     9m16s
ubuntu@ip-172-31-47-206:~$
```

11. Monitoring & Observability

The monitoring stack provides real-time visibility into the health and performance of the EC2 instance and the deployed application.

11.1 Node Exporter

Node Exporter is a Prometheus exporter that exposes hardware and OS-level metrics from the EC2 instance. It runs on port 9100 and serves metrics in Prometheus format.

```
Metrics URL: http://PUBLIC-IP:9100/metrics
```

11.2 Prometheus

Prometheus is a time-series database and monitoring system. It scrapes metrics from Node Exporter at regular intervals and stores them for querying and alerting.

```
Prometheus URL: http://PUBLIC-IP:9090
Targets URL:    http://PUBLIC-IP:9090/targets
```

Expected target status: node_exporter → UP

11.3 Grafana

Grafana connects to Prometheus as a data source and renders the metrics into an interactive, visual dashboard. The Node Exporter Full dashboard (ID: 1860) is automatically provisioned by Ansible.

```
Grafana URL:    http://PUBLIC-IP:3000
Username:      admin
Password:      admin
```

Dashboard Metrics Visible

- CPU Usage (%)
- RAM / Memory Usage
- Disk Usage and I/O
- Network Traffic (In/Out)
- System Load Average
- Server Uptime

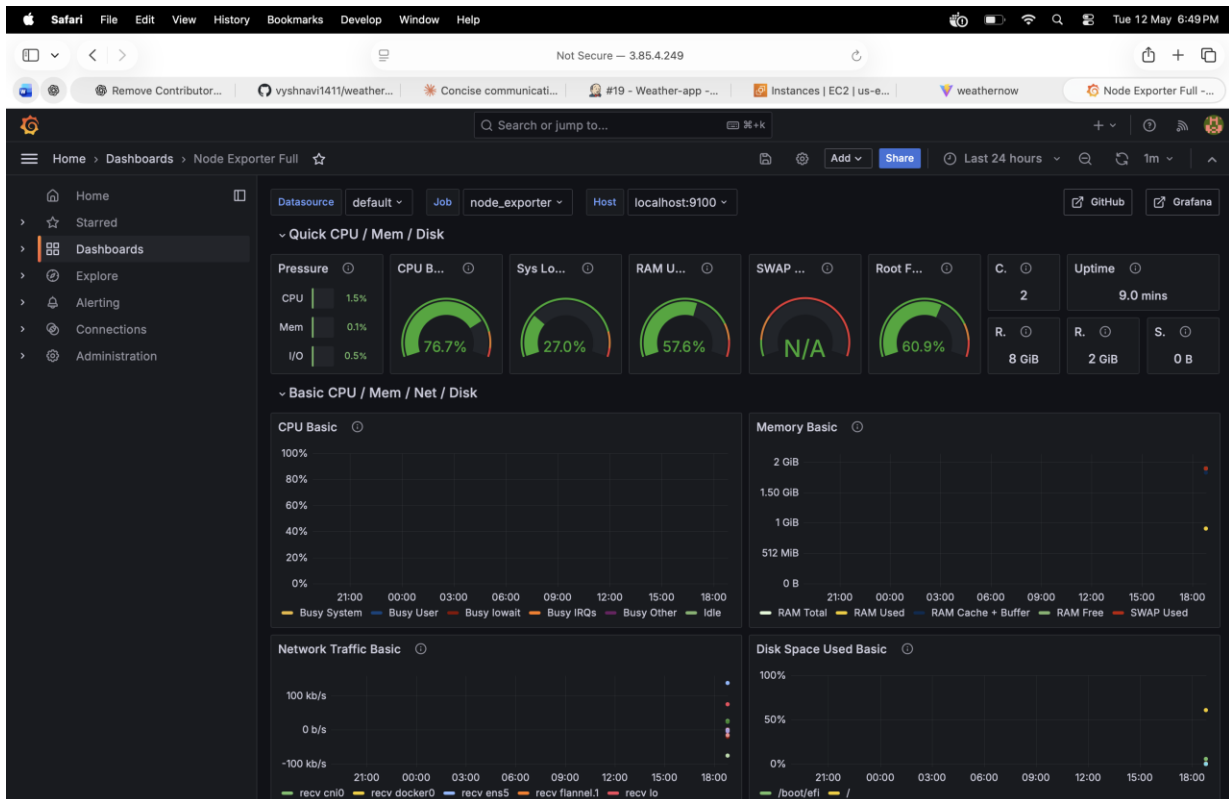
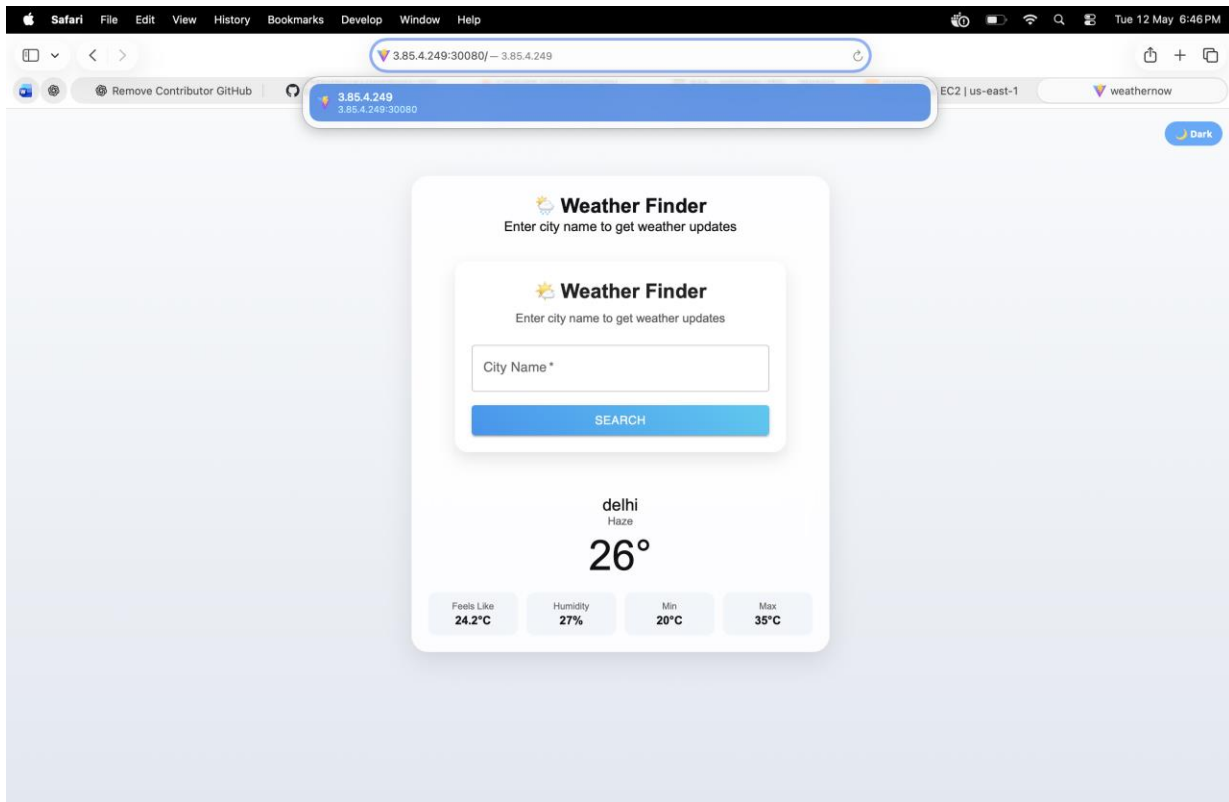
12. Verification & Access URLs

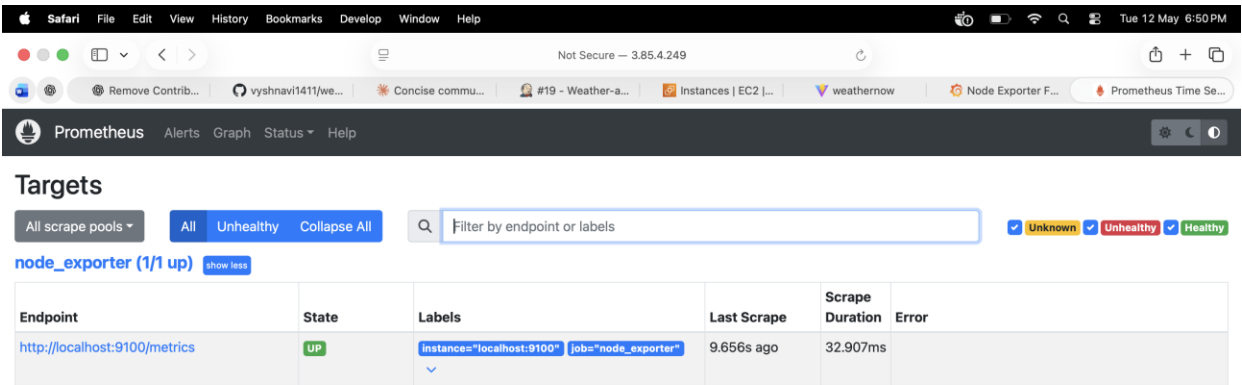
Once the Jenkins pipeline completes successfully, the following endpoints should be accessible:

Port	Service
http://PUBLIC-IP	Weather App (Nginx on port 80)
http://PUBLIC-IP:30080	Weather App (Kubernetes NodePort)
http://PUBLIC-IP:9090	Prometheus metrics server
http://PUBLIC-IP:9090/targets	Prometheus scrape targets (check node_exporter = UP)
http://PUBLIC-IP:3000	Grafana dashboard (admin / admin)
http://PUBLIC-IP:9100/metrics	Node Exporter raw metrics

The screenshot shows the Jenkins Pipeline Overview page for a project named 'Weather-app'. The pipeline is titled '#19' and is in the 'Pipeline Overview' view. The pipeline graph shows a sequence of steps: Start, Checkout SCM, Docker Build, Terraform Init, Terraform Validate, Terraform Plan, Terraform Apply, Generate Ansible..., Wait For EC2, Install Monitoring..., Verify Kubernetes, and Destroy Infrastructure. All steps are marked as successful with green checkmarks. The 'Destroy Infrastructure' step is currently selected, and a modal dialog is displayed asking 'Do you want to destroy infrastructure?' with 'Proceed' and 'Abort' buttons. The modal also shows the duration of the step as '60ms' and the time it started as '2m 47s ago'. The left sidebar lists the steps with their durations: Checkout SCM (1.1s), Docker Build (1m 8s), Terraform Init (2.3s), Terraform Validate (2.3s), Terraform Plan (3.8s), Terraform Apply (30s), Generate Ansible Inventory, Wait For EC2 (1m 8s), Install Monitoring & K3s Stack (6m), Verify Kubernetes (12s), and Destroy Infrastructure (60ms). The top of the page shows the Jenkins logo and the project name 'Weather-app'.

DevOps Weather App Deployment Project





13. Challenges & Solutions

Several real-world challenges were encountered during the development and testing of this project. Each was resolved through debugging, documentation research, and trial-and-error.

Challenge	Solution
Terraform path issues in Jenkins	Added Terraform binary path to Jenkins environment variables and used absolute paths in the Jenkinsfile.
GitHub webhook not triggering	Used ngrok to expose Jenkins publicly; verified payload URL and content type in GitHub settings.
SSH host verification blocking Ansible	Disabled strict host key checking in ansible.cfg using host_key_checking = False.
EC2 SSH key access denied	Ensured correct .pem key path was referenced in Ansible inventory and Jenkins credentials.
Prometheus not scraping Node Exporter	Fixed prometheus.yml target configuration to use the correct EC2 private IP.
Grafana datasource not auto-provisioning	Added provisioning YAML files for datasource and dashboard into Ansible playbook.

15. Conclusion

This project successfully demonstrates a complete, real-world DevOps automation workflow built entirely with industry-standard open-source tools and cloud infrastructure.

Starting from a simple React Weather Application, the project implements every layer of modern DevOps:

- Source control and collaboration with Git and GitHub
- Automated CI/CD pipeline with Jenkins and GitHub Webhooks
- Cloud infrastructure provisioning with Terraform and AWS EC2
- Agentless configuration management with Ansible
- Application containerization with Docker (multi-stage builds)
- Container orchestration with K3s (Kubernetes-compatible)
- Real-time monitoring and observability with Prometheus, Grafana, and Node Exporter

The entire deployment from a developer's Git push to a fully monitored, live application on the cloud is automated without any manual intervention. This makes the project a strong reference for DevOps learning, academic submissions, and professional portfolio demonstrations.